

TOP 10

機器學習演算法研讀報告

詳細說明、範例、圖文對照與實作導讀

本報告依據先前資訊圖表延伸，將十個常見機器學習演算法整理為可研讀、可教學、可作業引用的長篇教材。



TOP 10 機器學習演算法



#	演算法	類型	主要用途	圖示
1	Linear Regression 線性回歸	監督式學習	預測連續值 例如：房價預測	
2	Logistic Regression 邏輯回歸	監督式學習	二元分類 例如：垃圾郵件判斷	
3	Decision Tree 決策樹	監督式學習	易於理解的分類 / 回歸 例如：客戶是否會購買	
4	Random Forest 隨機森林	監督式學習	更準確的樹狀結構預測 例如：信用風險評估	
5	Support Vector Machine (SVM) 支持向量機	監督式學習	在高維空間中找到最佳分類邊界	
6	K-Nearest Neighbors (KNN) 近鄰演算法	監督式學習	根據鄰近資料點進行分類 例如：推薦系統	
7	Naive Bayes 樸素貝葉斯	監督式學習	常用於文字分類 例如：垃圾郵件過濾	
8	K-Means Clustering K-平均聚類	非監督式學習	將相似資料分群 例如：客戶分群	
9	Principal Component Analysis (PCA) 主成分分析	非監督式學習	降維 / 簡化特徵 保留重要資訊	
10	Neural Networks / Deep Learning 神經網路 / 深度學習	監督式 / 非監督式學習	影像、語音、NLP 等複雜的 AI 任務	

簡單分類總結



監督式學習 (有標準答案)

- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Naive Bayes



非監督式學習 (沒有標準答案)

- K-Means Clustering
- Principal Component Analysis (PCA)



深度學習

- Neural Networks / Deep Learning
- 包含 CNN、RNN、Transformer 等



選擇適合的演算法，能讓機器學習模型發揮最佳效果！

圖 0-1 Top 10 機器學習演算法總資訊圖表。

閱讀建議：先看總圖掌握分類，再依章節逐一研讀演算法原理與範例。

目錄

1. 一、報告目的與閱讀方式
2. 二、從問題型態看演算法分類
3. 三、機器學習流程與 CRISP-DM 觀念
4. 四、總覽表：十大演算法比較
5. 1. Linear Regression（線性迴歸）
6. 2. Logistic Regression（邏輯迴歸）
7. 3. Decision Tree（決策樹）
8. 4. Random Forest（隨機森林）
9. 5. Support Vector Machine, SVM（支持向量機）
10. 6. K-Nearest Neighbors, KNN（近鄰演算法）
11. 7. Naive Bayes（樸素貝葉斯）
12. 8. K-Means Clustering（K-平均聚類）
13. 9. Principal Component Analysis, PCA（主成分分析）
14. 10. Neural Networks / Deep Learning（神經網路／深度學習）
15. 附錄 A：如何選擇演算法
16. 附錄 B：評估指標與常見陷阱
17. 參考資料

註：此目錄為靜態閱讀目錄，便於快速定位章節；若需要自動頁碼目錄，可在 Word 中插入正式 TOC。

報告目的與閱讀方式

本研讀報告以「Top 10 機器學習演算法」為主軸，將先前資訊圖表中出現的十個演算法拆解成可研讀、可複習、可實作的章節。資訊圖表適合快速掌握全貌，但真正學會演算法，需要理解它在機器學習流程中的位置：資料從哪裡來、特徵如何表示、模型如何從資料中學習、預測結果如何評估、錯誤如何被解釋與修正。因此，本報告不只列名詞，也把每個演算法的核心直覺、公式意義、訓練流程、範例場景、優點限制、常見錯誤與入門 Python 寫法放在同一個章節中。讀者可以先看總表，再逐章閱讀，最後回到附錄的選模指南，練習判斷不同問題該用哪一類模型。

機器學習不是單純記住演算法名稱，而是學會把問題轉成資料與模型。舉例來說，若目標是預測房價，答案是一個連續數值，通常先考慮迴歸模型；若目標是判斷信件是否為垃圾郵件，答案是類別，通常先考慮分類模型；若沒有標準答案，只想找出客戶群組，則會轉向非監督式聚類；若資料是影像、語音、長文章、聊天內容，傳統表格模型可能不足，深度學習或大型語言模型的表徵能力會更重要。這種「先定義任務，再挑模型」的思考，比背誦模型清單更接近真實資料分析工作。

建議閱讀順序如下：第一輪先閱讀總覽與分類總結，建立監督式、非監督式、深度學習三大類的概念；第二輪依序閱讀線性迴歸、邏輯迴歸、決策樹、隨機森林、SVM、KNN、Naive Bayes，理解監督式學習如何從標準答案中學習；第三輪閱讀 K-Means 與 PCA，理解沒有標準答案時如何探索資料結構；第四輪閱讀神經網路，建立深度學習與傳統機器學習的關係。最後可用附錄中的檢核表，把自己的資料問題對照到適合的演算法。

從問題型態看演算法分類

機器學習演算法通常依照學習訊號分成監督式學習、非監督式學習與其他延伸類型。監督式學習的重點是資料中已有標準答案，例如房價、是否購買、是否詐欺、客戶是否流失。模型的任務是在輸入特徵 X 與目標 y 之間建立映射，未來看到新資料時能給出合理預測。線性迴歸、邏輯迴歸、決策樹、隨機森林、SVM、KNN 與 Naive Bayes 都常被放在監督式學習範圍內。

非監督式學習沒有明確標準答案，目的通常是探索資料內部結構。K-Means 會嘗試把相似資料分到同一群，PCA 則會把高維特徵轉成較少的主成分，協助視覺化、壓縮或降低雜訊。這類方法常出現在資料探索階段，幫助分析者理解資料是否存在自然群組、重要方向或異常樣本。非監督式學習的結果通常不應被視為最終真理，而是探索工具，需要搭配商業知識與後續驗證。

深度學習可以視為機器學習的一個重要分支，它使用多層神經網路表示複雜函數。與傳統模型相比，深度學習特別擅長處理非結構化資料，例如圖片、聲音、文字與影片。CNN、RNN、Transformer 等模型都屬於神經網路家族。它的優勢是表徵能力強，限制是需要較多資料、算力、訓練技巧與解釋工具。對初學者而言，先理解前九個傳統演算法，再看神經網路，會更容易看懂深度學習其實是在做更複雜的特徵轉換與函數逼近。

機器學習流程與 CRISP-DM 觀念

在實務專案中，演算法只是流程的一部分。以 CRISP-DM 思維來看，第一步不是開啟 Python，而是理解商業問題：要解決什麼痛點？預測結果會用在哪裡？錯判成本是什麼？第二步是理解資料：資料欄位代表什麼、品質如何、有無缺漏、有無偏差。第三步是資料準備：清理、轉換、編碼、標準化、切分訓練集與測試集。第四

步才是建模：挑選適合的演算法並訓練。第五步是評估：不能只看訓練集分數，還要看測試資料、交叉驗證、錯誤案例與業務指標。第六步是部署與監控：模型上線後資料分布可能變動，必須持續觀察效果。

這份報告在每個演算法章節都會加入實作與評估提醒，原因是許多初學者常犯的錯不在公式，而在流程。例如，把測試資料拿去做標準化 fit，會造成資料洩漏；只看 accuracy，可能忽略類別不平衡；只追求複雜模型，可能失去可解釋性；只看模型分數，卻沒有思考錯判成本，會讓模型難以落地。真正的機器學習能力，是把資料、模型、評估、溝通與應用場景連在一起。

若要把本報告用於作業或專題，可以用以下邏輯規劃：先選一個情境，例如房價預測、客戶流失、商品推薦、垃圾郵件分類、客戶分群；接著說明資料欄位與目標；再選擇至少兩種演算法比較，例如線性迴歸與隨機森林，或邏輯迴歸與 SVM；最後用圖表呈現資料分布、模型結果與錯誤分析。這樣的報告會比單純列出模型名稱更像完整資料科學專案。



圖 0-2 資訊圖表下方分類總結：監督式、非監督式與深度學習。

總覽表：十大演算法比較

下表將十個演算法以任務型態、核心直覺與常見用途進行整理。這張表適合在正式學習前建立快速索引，也適合在專題初期用來挑選 baseline 模型。

#	演算法	類型	核心直覺	常見用途
1	Linear Regression 線性迴歸	監督式學習 / 回歸	用一條直線或超平面描述特徵與連續數值目標之間的平均關係。	連續數值預測、趨勢估計、基準模型、商業指標預測
2	Logistic Regression 邏輯迴歸	監督式學習 / 分類	把線性分數轉換為 0 到 1 的機率，常用於二元分類與可解釋分類。	二元分類、風險評分、轉換率預測、詐欺偵測初版模型
3	Decision Tree 決策樹	監督式學習 / 分類與回歸	像問卷一樣一層層切分資料，從根節點走到葉節點得到預測。	規則萃取、可視化分類、可解釋模型、混合型資料探索
4	Random Forest 隨機森林	監督式學習 / 集成學習	訓練許多不完全相同的決策樹，再把它們的結果合併，以降低單棵樹的不穩定。	表格資料分類與回歸、特徵重要性分析、非線性預測、強 baseline
5	Support Vector Machine, SVM 支持向量機	監督式學習 / 分類與回歸	在資料點之間找出一條間隔最大的分隔線或超平面，邊界由支持向量決定。	高維分類、中小型資料、清楚邊界問題、文字特徵分類
6	K-Nearest Neighbors, KNN 近鄰演算法	監督式學習 / 分類與回歸	不急著學出複雜公式，而是在預測時查看新資料附近的已知資料。	小型資料分類、相似度搜尋、推薦雛形、局部模式判斷

7	Naive Bayes 樸素貝葉斯	監督式學習 / 機率分類	以貝葉斯定理估計類別機率，並假設特徵在給定類別後彼此條件獨立。	文字分類、垃圾郵件過濾、快速基準模型、串流或大規模稀疏特徵
8	K-Means Clustering K-平均聚類	非監督式學習 / 聚類	事先指定群數 K，讓每個資料點分配到最近的群中心，再反覆更新中心。	分群探索、客戶分群、影像色彩壓縮、資料前處理
9	Principal Component Analysis, PCA 主成分分析	非監督式學習 / 降維	把多個相關特徵轉換為少數主成分，用較少維度保留大部分資訊。	降維、視覺化、去除共線性、資料壓縮、模型前處理
10	Neural Networks / Deep Learning 神經網路／深度學習	監督式與非監督式皆可	利用多層節點與非線性轉換，從大量資料中學習複雜表徵。	影像、語音、自然語言、生成式 AI、複雜非結構化資料

表 0-1 十大機器學習演算法比較表。

1. Linear Regression（線性迴歸）

章節定位：監督式學習 / 回歸。核心概念：用一條直線或超平面描述特徵與連續數值目標之間的平均關係。代表公式／規則： $\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_px_p$



圖 1-1 資訊圖表細部裁切：Linear Regression（線性迴歸）。

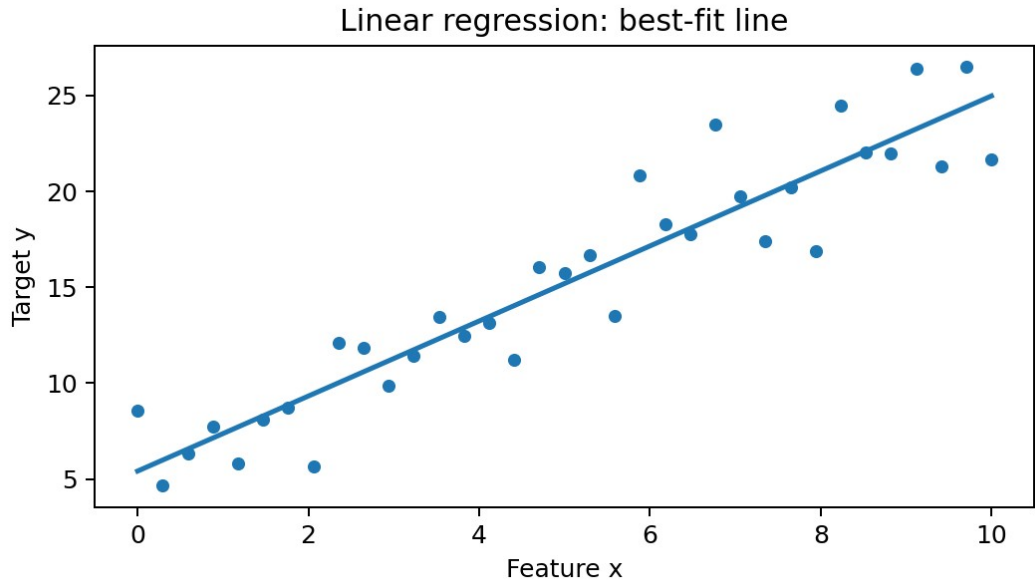


圖 1-2 線性迴歸概念示意圖。

核心直覺與學習目標

線性迴歸的任務可以簡化理解為：用一條直線或超平面描述特徵與連續數值目標之間的平均關係。它常見用途包括：連續數值預測、趨勢估計、基準模型、商業指標預測。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

線性迴歸是許多機器學習課程的起點，因為它把「用資料找出一條最佳趨勢線」這件事講得非常清楚。假設我們用坪數預測房價，模型會嘗試找到一條線，使每筆資料的真實價格與預測價格差距盡量小。若特徵只有一個，圖上看到的是二維平面中的一條直線；若特徵很多，模型其實是在高維空間中找一個超平面。這個超平面雖然不容易直接畫出來，但概念仍然相同：每個特徵都有一個權重，權重代表該特徵改變時，預測值平均會如何改變。

線性迴歸最常使用的訓練目標是最小化平方誤差，也就是讓預測值與真實值的差距平方總和最小。平方誤差會放大大錯誤，因此模型會努力避免少數資料點偏離太多。這個特性既是優點也是限制：它能让模型對大錯誤敏感，但也讓模型容易受離群值影響。若資料中有非常異常的成交價，例如極端豪宅或資料輸入錯誤，迴歸線可能被拉偏。因此，在實務中使用線性迴歸前，常要先畫散佈圖、檢查離群值、思考是否需要對目標做 log 轉換。

線性迴歸的可解釋性很高。假設模型包含坪數、屋齡、距離捷運站三個特徵，係數可以告訴我們：在其他條件不變時，坪數增加一單位，房價預測平均增加多少；屋齡增加一單位，房價預測平均改變多少。這種解釋很適合商業溝通，也適合作為複雜模型的比較基準。即使最後使用隨機森林或神經網路，線性迴歸仍能提供一個透明的 baseline，幫助我們確認複雜模型是否真的帶來價值。

不過，線性迴歸有一個重要假設：目標與特徵之間大致可用線性組合描述。現實世界常有非線性，例如廣告投入與銷售額可能一開始成長很快，但投入過多後邊際效果下降；屋齡對房價的影響可能不是固定斜率；收入與消費可能存在門檻。遇到這些情況，可以加入多項式特徵、交互作用特徵，或改用決策樹、隨機森林、梯度提升等模型。

學習線性迴歸時，建議特別注意三件事：第一，資料切分要正確，不能用測試資料參與訓練；第二，評估指標要符合任務，常見有 MAE、MSE、RMSE、 R^2 ；第三，模型殘差要檢查，因為殘差圖可以揭露非線性、變異不均或離群值。若只是執行 fit 與 predict 而不看殘差，就容易錯過模型失效的訊號。

範例情境

房價預測：輸入坪數、屋齡、距離捷運站、樓層，輸出預估成交價。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 模型簡單且容易解釋，係數可以直接看出特徵方向與相對影響。
- 訓練速度快，適合作為回歸任務的第一個 baseline。
- 在資料關係接近線性、特徵工程適當時，常能提供穩定且透明的結果。

限制與注意事項

- 無法自然捕捉高度非線性關係，需要多項式特徵或其他模型補強。
- 容易受到離群值影響；極端資料可能拉動迴歸線。
- 特徵高度共線時，係數解讀容易不穩定。

Python 入門範例

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把線性迴歸寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用線性迴歸」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把線性迴歸與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，線性迴歸不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

2. Logistic Regression（邏輯迴歸）

章節定位：監督式學習 / 分類。核心概念：把線性分數轉換為 0 到 1 的機率，常用於二元分類與可解釋分類。代表公式／規則： $P(y=1|x) = 1 / (1 + e^{-(z)})$, $z = b + wx$



圖 2-1 資訊圖表細部裁切：Logistic Regression（邏輯迴歸）。

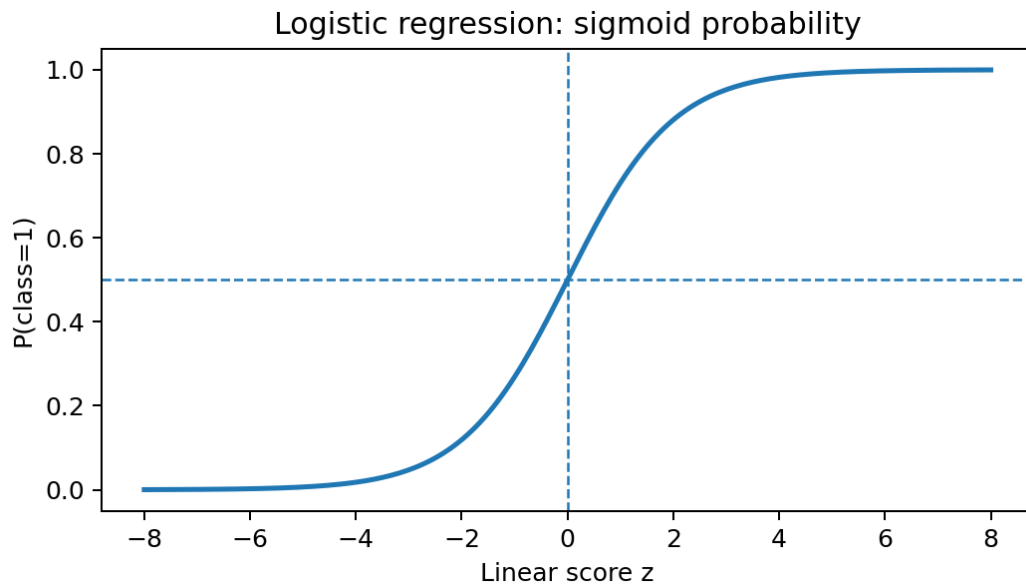


圖 2-2 邏輯迴歸概念示意圖。

核心直覺與學習目標

邏輯迴歸的任務可以簡化理解為：把線性分數轉換為 0 到 1 的機率，常用於二元分類與可解釋分類。它常見用途包括：二元分類、風險評分、轉換率預測、詐欺偵測初版模型。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

邏輯迴歸雖然名稱中有「迴歸」，但最常用於分類，尤其是二元分類。它先像線性迴歸一樣計算一個線性分數 z ，再透過 sigmoid 函數把分數壓到 0 到 1 之間，解讀為屬於正類的機率。例如在垃圾郵件判斷中，模型可以輸出 0.92，表示這封信很可能是垃圾郵件；在客戶流失預測中，輸出 0.73 表示該客戶流失風險較高。

邏輯迴歸的價值在於它同時兼具簡潔、速度與可解釋性。每個特徵的係數代表該特徵對 log-odds 的影響，進一步可以解讀為風險增加或降低的方向。這對金融風控、醫療風險、行銷轉換率等場景很有用，因為決策者通常不只想知道預測答案，也想知道模型為何這樣判斷。若某些特徵係數過大或方向違反常識，就可能提醒我們資料有偏差、欄位定義錯誤或樣本分布不合理。

邏輯迴歸的訓練通常以最大概似估計或最小化交叉熵損失為目標。直覺上，若真實類別是 1，模型希望輸出機率接近 1；若真實類別是 0，模型希望輸出機率接近 0。錯得越有信心，懲罰越大。這和單純計算分類對錯不同，因為機率本身也被納入訓練。也因此，邏輯迴歸很適合需要排序的問題，例如把客戶依流失風險由高到低排列，讓客服優先聯繫高風險客戶。

使用邏輯迴歸時，不能把 0.5 門檻視為永遠正確。若偵測詐欺交易，漏掉詐欺的成本可能很高，門檻可以降低以提高召回率；若錯誤封鎖正常交易成本很高，門檻就要提高以降低誤報。分類任務應該搭配 confusion matrix、precision、recall、F1、ROC-AUC、PR-AUC 等指標，而不是只看 accuracy。特別是類別不平衡問題，例如詐欺交易只有 1%，模型全部猜正常也有 99% accuracy，但完全沒有實用價值。

邏輯迴歸也有局限。它的決策邊界通常是線性的，對複雜非線性問題可能不夠。特徵尺度差異很大時，若有正則化，標準化會影響係數與效果。資料中若存在強烈交互作用，可能需要人工加入交互特徵，或改用樹模型。

即便如此，它仍然是分類任務中非常重要的基準模型，因為它快速、穩定、可解釋，而且能建立對分類機率的直覺。

範例情境

垃圾郵件判斷：輸入信件文字特徵、寄件者、連結數量，輸出是否為垃圾郵件的機率。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 輸出是機率，適合風險排序與門檻調整。
- 係數可解釋，能說明某特徵提高或降低正類機率。
- 搭配正則化後，能處理多特徵且降低過度擬合。

限制與注意事項

- 分類邊界本質上偏線性，複雜非線性資料需特徵工程或改用其他模型。
- 對特徵尺度、離群值與類別不平衡仍需謹慎處理。
- 機率校準與決策門檻必須依業務成本調整，不能只看 0.5。

Python 入門範例

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
prob = model.predict_proba(X_test)[: , 1]
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把邏輯迴歸寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用邏輯迴歸」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把邏輯迴歸與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

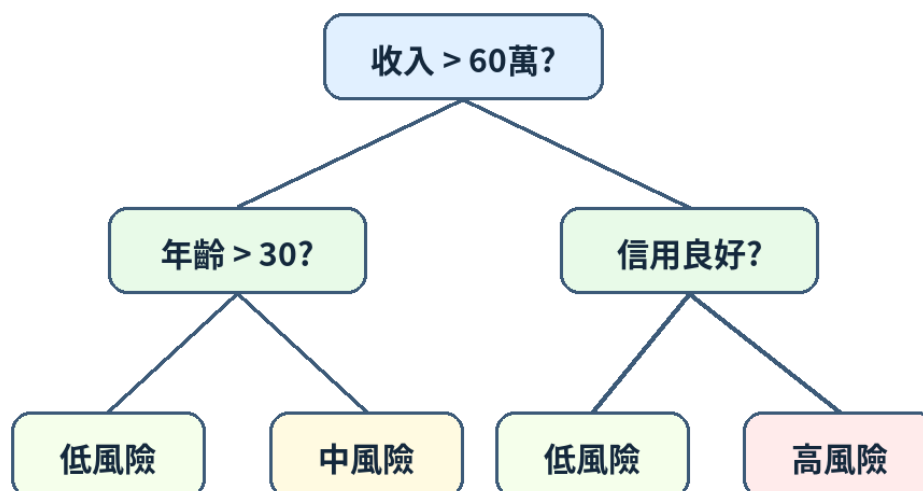
從學習角度看，邏輯迴歸不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

3. Decision Tree（決策樹）

章節定位：監督式學習 / 分類與回歸。核心概念：像問卷一樣一層層切分資料，從根節點走到葉節點得到預測。代表公式／規則：以特徵切分使節點 impurity 下降，例如 Gini、Entropy、MSE。



圖 3-1 資訊圖表細部裁切：Decision Tree（決策樹）。



Decision tree: 每個節點是一個問題，路徑最後通向預測結果。

圖 3-2 決策樹概念示意圖。

核心直覺與學習目標

決策樹的任務可以簡化理解為：像問卷一樣一層層切分資料，從根節點走到葉節點得到預測。它常見用途包括：規則萃取、可視化分類、可解釋模型、混合型資料探索。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

決策樹可以想像成一套自動產生的判斷規則。模型會在每個節點選擇一個特徵與切分條件，例如「收入是否大於 60 萬」、「近 30 天是否瀏覽商品」、「信用分數是否高於某門檻」。資料沿著不同分支往下走，最後到達葉節點，葉節點中的多數類別或平均值就是預測結果。這種形式非常貼近人類決策，因此常被用於教學與初步資料探索。

決策樹的核心在於如何選擇切分。分類樹常用 Gini impurity 或 entropy 衡量節點純度，迴歸樹則常用平均平方誤差。模型會尋找能最大程度降低不純度的切分方式。直覺上，一個好的切分應該讓分出去的子節點更純，例如左邊大多是會購買的客戶，右邊大多是不會購買的客戶。若切分後兩邊仍然混雜，代表該條件沒有提供太多資訊。

決策樹的優點是能處理非線性與特徵交互作用。假設一個客戶是否購買，不只取決於收入，也取決於是否看過促銷、是否是老客戶、是否對價格敏感。線性模型需要額外設計交互特徵，決策樹則能自然地用多層切分表達「若 A 且 B，則結果不同」的邏輯。此外，決策樹不需要特徵標準化，因為它只關心排序與切分門檻，不關心距離尺度。

然而，單棵決策樹很容易過度擬合。若讓樹一直長下去，它可能為了完美分類訓練資料而建立非常細碎的規則，導致新資料表現變差。常見控制方式包括限制 max_depth、min_samples_split、min_samples_leaf，或進行剪枝。可以把決策樹想成一本規則手冊：規則太少會過於粗糙，規則太多則可能只是在記住過去案例，而不是學到可泛化的模式。

在報告或簡報中，決策樹很適合用來說明模型如何做判斷。你可以畫出前三層樹，展示最重要的特徵與分支邏輯。但若追求最高準確率，通常會進一步使用隨機森林或梯度提升樹。這些集成模型犧牲部分可解釋性，換取更穩定的預測。學習決策樹的重點，不只是會呼叫模型，而是理解資料切分、過度擬合與模型複雜度之間的關係。

範例情境

客戶是否購買：先問是否曾瀏覽商品，再問價格敏感度、會員等級、近期活動互動。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 規則很直覺，非技術人員也容易理解。
- 能處理非線性與特徵交互作用，不必先假設關係是直線。
- 對特徵尺度不敏感，通常不需要標準化。

限制與注意事項

- 單棵樹容易過度擬合，深度太大時會記住訓練資料細節。
- 資料稍微變動，樹的結構可能大幅改變。
- 在連續邊界光滑的問題上，預測結果可能呈階梯狀。

Python 入門範例

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把決策樹寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用決策樹」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把決策樹與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

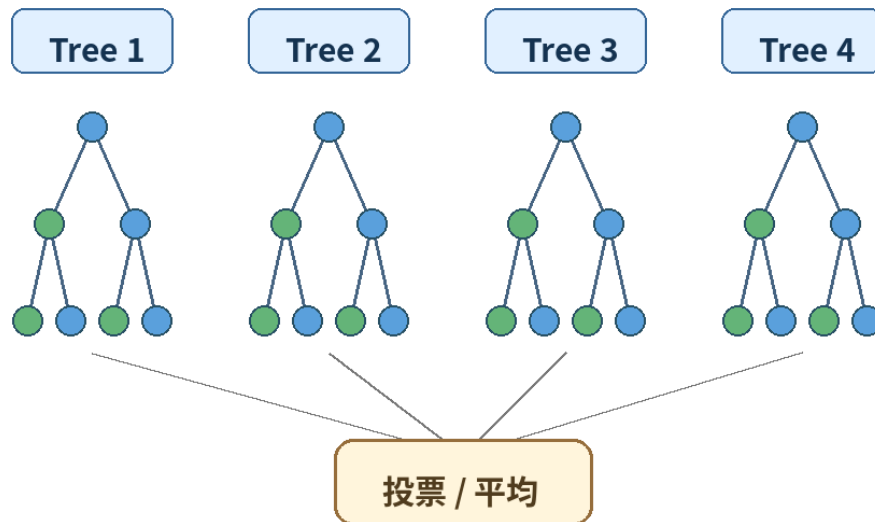
從學習角度看，決策樹不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

4. Random Forest (隨機森林)

章節定位：監督式學習 / 集成學習。核心概念：訓練許多不完全相同的決策樹，再把它們的結果合併，以降低單棵樹的不穩定。代表公式／規則： $\hat{y} = \text{aggregate}(\text{tree1}(x), \dots, \text{treeB}(x))$



圖 4-1 資訊圖表細部裁切：Random Forest (隨機森林)。



Random forest: 多棵樹用不同樣本與特徵訓練，最後投票或平均，降低單一樹過度

圖 4-2 隨機森林概念示意圖。

核心直覺與學習目標

隨機森林的任務可以簡化理解為：訓練許多不完全相同的決策樹，再把它們的結果合併，以降低單棵樹的不穩定。它常見用途包括：表格資料分類與回歸、特徵重要性分析、非線性預測、強 baseline。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

隨機森林是由許多決策樹組成的集成模型。它的基本想法很像詢問多位專家，而不是只相信一個人的判斷。每棵樹使用不同的抽樣資料與不同的特徵子集合訓練，因此每棵樹會學到略微不同的規則。分類時，森林讓多棵樹投票；回歸時，森林取多棵樹預測值的平均。這種做法可以降低單棵樹不穩定與過度擬合的問題。

隨機森林常被視為表格資料的強 baseline。它不需要太多特徵縮放，不需要明確指定線性關係，也能處理非線性與特徵交互作用。對初學者而言，若遇到分類或回歸的表格資料，又不確定該用哪個模型，隨機森林通常是一個值得嘗試的選項。它常能在很少特徵工程的情況下取得不錯表現，並且能提供特徵重要性，協助理解哪些欄位對模型較有貢獻。

隨機森林中的「隨機」有兩個層次。第一是 bootstrap sampling，也就是每棵樹從原始資料中有放回抽樣，產生不同的訓練集；第二是在每個節點切分時，不是看全部特徵，而是隨機選一部分特徵來尋找最佳切分。這兩種隨機性能降低樹與樹之間的相似度，使投票或平均更有效。若所有樹都一模一樣，投票就沒有意義；若每棵樹錯誤方向不同，集成後能互相抵消。

使用隨機森林時，常見參數包括 `n_estimators`、`max_depth`、`min_samples_leaf`、`max_features`。`n_estimators` 代表樹的數量，增加通常能提升穩定度，但也增加訓練與預測時間。`max_depth` 與 `min_samples_leaf` 控制每棵樹的複雜度，避免太深導致過度擬合。實務中，隨機森林雖然比單棵樹穩定，但仍然不是萬能。若資料有時間序列趨勢、需要外推到訓練範圍外，或特徵中有高度稀疏文字，可能要考慮其他模型。

隨機森林的可解釋性介於決策樹與黑箱模型之間。單棵樹很好解釋，但森林由大量樹組成，不容易逐條規則說明。不過，可以透過特徵重要性、Permutation Importance、Partial Dependence Plot、SHAP 等方法輔助

理解。對作業或專題來說，一個很好的呈現方式是：先用決策樹展示簡單規則，再用隨機森林提升準確率，最後比較兩者在解釋性與預測效果上的差異。

範例情境

信用風險評估：多棵樹從不同樣本與特徵角度判斷違約風險，最後整合為穩定分數。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 通常比單棵決策樹更準、更穩定。
- 能處理非線性、特徵交互與混合型資料。
- 可產生特徵重要性，利於資料理解與模型溝通。

限制與注意事項

- 比單棵樹難解釋，整體決策不是一條簡單規則。
- 模型體積與預測成本較高。
- 若資料外推到訓練範圍以外，樹模型通常不擅長線性外推。

Python 入門範例

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=300, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把隨機森林寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用隨機森林」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把隨機森林與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，隨機森林不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

5. Support Vector Machine, SVM（支持向量機）

章節定位：監督式學習 / 分類與回歸。核心概念：在資料點之間找出一條間隔最大的分隔線或超平面，邊界由支持向量決定。代表公式／規則：尋找最大間隔分類邊界；非線性時可使用 kernel trick。



圖 5-1 資訊圖表細部裁切：Support Vector Machine, SVM（支持向量機）。

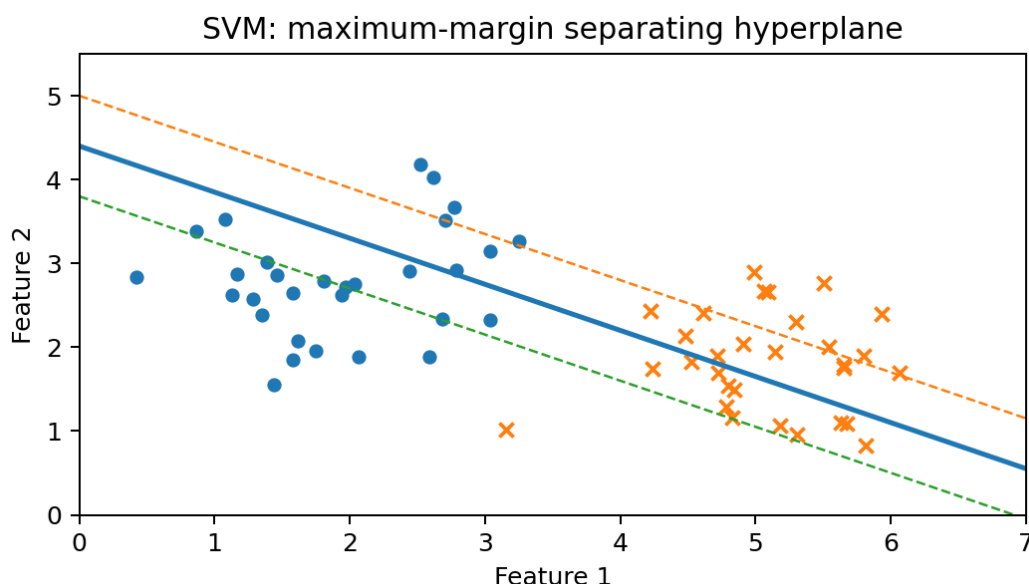


圖 5-2 支持向量機概念示意圖。

核心直覺與學習目標

支持向量機的任務可以簡化理解為：在資料點之間找出一條間隔最大的分隔線或超平面，邊界由支持向量決定。它常見用途包括：高維分類、小中型資料、清楚邊界問題、文字特徵分類。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

支持向量機的核心直覺是尋找一條分類邊界，讓不同類別之間的間隔最大。以二維資料為例，若兩群點可以被一條線分開，SVM 不只要找能分開的線，而是要找離兩邊最近資料點距離最大的線。這些最靠近邊界、真正決定分類線位置的資料點稱為支持向量。也就是說，SVM 的決策邊界主要由少數關鍵樣本決定。

SVM 的優勢之一是高維資料。在文字分類中，每個詞都可能是一個特徵，特徵數可能比樣本數多很多。SVM 在這類情境常有良好表現，尤其是線性 SVM 搭配 TF-IDF 特徵。它不需要直接理解語意，只要在高維特徵空間中找到能區分類別的超平面，就能完成主題分類、情緒分類或垃圾郵件偵測。這也是為什麼在深度學習普及前，SVM 曾是許多文本分類任務的強力方法。

非線性問題中，SVM 可使用 kernel trick，把資料隱含映射到更高維空間，使原本不可線性分割的資料在新空間中變得可分。常見 kernel 有 linear、polynomial、RBF。RBF kernel 很常用，但參數 gamma 與 C 需要調整。C 控制錯誤容忍與間隔大小的權衡；gamma 控制單一樣本影響範圍。若 gamma 太大，模型容易只記住局部資料；若太小，邊界可能過度平滑。

使用 SVM 時，特徵標準化非常重要。因為 SVM 基於距離與內積計算，若某個特徵尺度遠大於其他特徵，它會主導模型。通常會用 StandardScaler 建立 pipeline，確保訓練與測試資料使用同一套轉換。對初學者來說，若 SVM 效果不好，第一件事常常不是換模型，而是確認是否標準化、是否調整 C 與 gamma、是否選對 kernel。

SVM 的限制包括大型資料訓練成本較高、參數選擇敏感、結果解釋不如線性模型直覺。若資料筆數非常大，可能改用 LinearSVC、SGDClassifier 或其他更可擴展方法。若業務需要清楚說明每個決策理由，SVM 也可能不如邏輯迴歸或決策樹容易溝通。總結來說，SVM 適合邊界清楚、小中型資料、高維特徵的分類任務，尤其適合作為傳統機器學習中理解「間隔」與「核函數」概念的重要模型。

範例情境

小型高維文字分類：用 TF-IDF 特徵表示文章，判斷主題或情緒類別。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 在高維空間中表現常很好，尤其是特徵數大於樣本數時。
- 支持向量使模型只依賴部分關鍵樣本點。
- 透過 kernel 可建立非線性分類邊界。

限制與注意事項

- 大型資料訓練成本可能高，參數 C、gamma、kernel 需要調整。
- 機率輸出不是天然結果，若需要機率需額外校準。
- 對特徵尺度敏感，通常要標準化。

Python 入門範例

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
model = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1.0, gamma='scale'))
```

```
model.fit(X_train, y_train)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把支持向量機寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用支持向量機」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把支持向量機與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，支持向量機不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

6. K-Nearest Neighbors, KNN（近鄰演算法）

章節定位：監督式學習 / 分類與回歸。核心概念：不急著學出複雜公式，而是在預測時查看新資料附近的已知資料。代表公式／規則：找到距離最近的 k 個鄰居，分類用多數決，回歸用平均。



圖 6-1 資訊圖表細部裁切：K-Nearest Neighbors, KNN（近鄰演算法）。

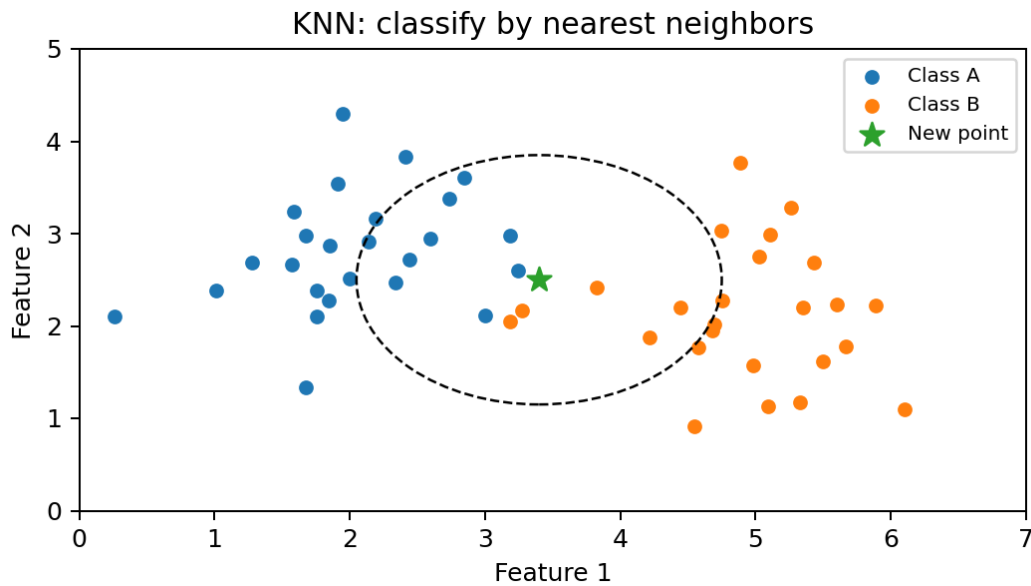


圖 6-2 近鄰演算法概念示意圖。

核心直覺與學習目標

近鄰演算法的任務可以簡化理解為：不急著學出複雜公式，而是在預測時查看新資料附近的已知資料。它常見用途包括：小型資料分類、相似度搜尋、推薦雛形、局部模式判斷。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

KNN 是一個非常直覺的演算法：新資料的類別由它附近的資料決定。假設 $k=5$ ，模型會找到離新資料最近的 5 個訓練樣本，若其中 3 個是 A 類、2 個是 B 類，就預測為 A 類。若是回歸問題，則可以取鄰居目標值的平均。KNN 幾乎不需要明確訓練，因為它的知識主要保存在訓練資料本身，真正的計算發生在預測時。

KNN 的關鍵是距離。常見距離有歐氏距離、曼哈頓距離、餘弦距離等。距離的意義必須符合問題情境。例如用身高、體重、年齡判斷類別時，若不標準化，數值範圍大的特徵會主導距離；用文字向量時，餘弦相似度可能比歐氏距離更有意義。這提醒我們，KNN 看似簡單，實際上非常依賴特徵表示與尺度處理。

k 值會影響模型複雜度。 k 很小時，模型對局部變化很敏感，可能受噪音影響； k 很大時，模型會過度平滑，忽略局部差異。可以把 k 想成鄰居投票的範圍：問太少人，答案容易被偶然個案影響；問太多人，答案又可能變得太平均。實務上常用交叉驗證選擇 k ，也會測試是否使用距離加權，讓越近的鄰居權重越高。

KNN 適合教學、小型資料、局部模式明顯的問題，也常被用於相似度搜尋與推薦雛形。例如，若要做一個簡單推薦系統，可以用使用者行為特徵計算相似度，找出最相似的使用者，再推薦他們喜歡而目標使用者尚未接觸的商品。雖然大型推薦系統會使用更複雜方法，但 KNN 能幫助初學者理解「相似的人可能喜歡相似的東西」這個基礎概念。

KNN 的限制是預測成本高與維度災難。資料筆數越多，找最近鄰居越慢；特徵維度越高，資料點之間的距離差異可能變得不明顯，使鄰近概念失去效果。因此，高維資料常需要先做特徵選擇、PCA 降維，或改用適合高維稀疏資料的方法。學 KNN 的重點不是把它當成最強模型，而是用它理解距離、相似度、尺度與局部決策。

範例情境

推薦系統入門：找到與某位使用者行為最相近的 k 位使用者，再推薦他們喜歡的項目。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 概念簡單，容易實作與教學。
- 能捕捉局部非線性結構。
- 幾乎沒有訓練成本，新增資料很直覺。

限制與注意事項

- 預測時要計算距離，資料量大時成本高。
- 對特徵尺度非常敏感，通常必須標準化。
- 高維資料中距離容易失去判別力，稱為維度災難。

Python 入門範例

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把近鄰演算法寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用近鄰演算法」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把近鄰演算法與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，近鄰演算法不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言

說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

7. Naive Bayes（樸素貝葉斯）

章節定位：監督式學習 / 機率分類。核心概念：以貝葉斯定理估計類別機率，並假設特徵在給定類別後彼此條件獨立。代表公式／規則： $P(y|x_1, \dots, x_n) \propto P(y) \times \prod P(x_i|y)$



圖 7-1 資訊圖表細部裁切：Naive Bayes（樸素貝葉斯）。

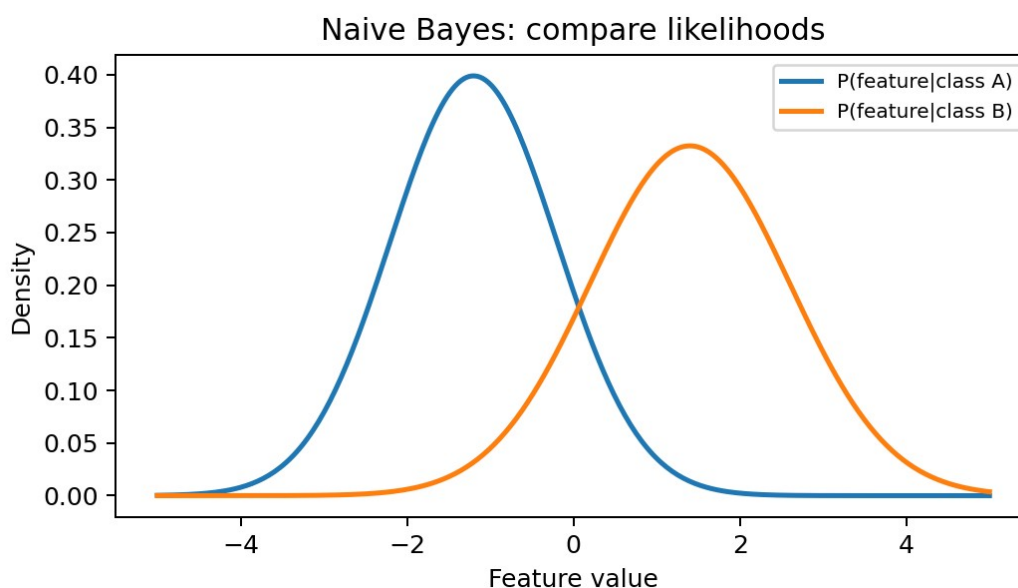


圖 7-2 樸素貝葉斯概念示意圖。

核心直覺與學習目標

樸素貝葉斯的任務可以簡化理解為：以貝葉斯定理估計類別機率，並假設特徵在給定類別後彼此條件獨立。它常見用途包括：文字分類、垃圾郵件過濾、快速基準模型、串流或大規模稀疏特徵。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

樸素貝葉斯建立在貝葉斯定理之上，目標是根據觀察到的特徵，估計每個類別的後驗機率。它被稱為「樸素」，是因為它假設在給定類別後，各特徵彼此條件獨立。這個假設在現實中通常不完全成立，例如文字中的詞彙常有關聯，但令人驚訝的是，Naive Bayes 在許多文字分類問題上仍然很有效。

以垃圾郵件分類為例，模型會估計某些詞在垃圾郵件與正常郵件中出現的機率。若信件中出現「免費」、「中獎」、「立即領取」等詞，模型會計算這些詞在垃圾郵件類別中的可能性，並與正常郵件類別比較。最後選擇

後驗機率最大的類別。Multinomial Naive Bayes 特別適合詞頻特徵，Bernoulli Naive Bayes 適合詞是否出現的二元特徵，Gaussian Naive Bayes 則常用於連續數值特徵。

Naive Bayes 的速度非常快，訓練資料量需求也相對低。它常被用作文字分類 baseline，原因是文字特徵通常非常高維且稀疏，傳統複雜模型未必一開始就穩定，而 Naive Bayes 能快速給出可比較結果。在實務專案中，先用 Naive Bayes 建立一個簡單可用版本，再比較邏輯迴歸、SVM 或深度學習模型，是合理的開發流程。

使用 Naive Bayes 時，要理解平滑的重要性。若某個詞在某類訓練資料中從未出現，沒有平滑時機率可能變成 0，導致整個乘積為 0。Laplace smoothing 會為每個特徵加入小量計數，避免零機率問題。這是一個很好的例子，說明機率模型在數學上看似簡單，但實作細節會直接影響效果。

Naive Bayes 的限制是獨立假設與表徵能力。它不擅長捕捉詞序、語境、反諷或長距離語意關係。如果任務需要理解文章深層語意，Transformer 類模型可能更適合。但在資源有限、需要快速、資料稀疏、可解釋需求高的場景，Naive Bayes 仍然是非常值得掌握的演算法。它也能幫助初學者理解先驗機率、似然、後驗機率與分類決策之間的關係。

範例情境

文字分類：根據文章中詞彙出現機率，判斷信件是垃圾郵件、新聞是體育或財經。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 訓練與預測速度很快，適合大量文字特徵。
- 即使獨立假設不完全成立，實務上仍常有不錯效果。
- 需要的資料量相對較少，是文字分類常用 baseline。

限制與注意事項

- 特徵獨立假設過強，若特徵高度相關可能影響機率解讀。
- 零機率問題需要平滑處理，例如 Laplace smoothing。
- 若需要捕捉語序、語意或長距依賴，傳統 Naive Bayes 能力有限。

Python 入門範例

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
model = make_pipeline(CountVectorizer(), MultinomialNB())
model.fit(text_train, y_train)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把樸素貝葉斯寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用樸素貝葉斯」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把樸素貝葉斯與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

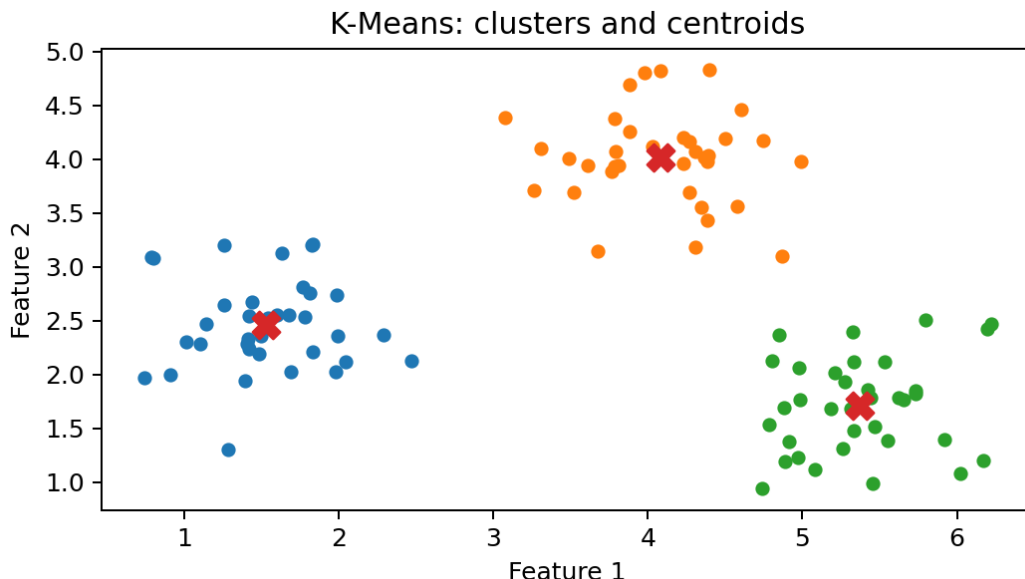
從學習角度看，樸素貝葉斯不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

8. K-Means Clustering (K-平均聚類)

章節定位：非監督式學習 / 聚類。核心概念：事先指定群數 K，讓每個資料點分配到最近的群中心，再反覆更新中心。代表公式／規則：最小化群內平方和： $\sum \min ||x_i - \mu_j||^2$



圖 8-1 資訊圖表細部裁切：K-Means Clustering (K-平均聚類)。



核心直覺與學習目標

K-平均聚類的任務可以簡化理解為：事先指定群數 K ，讓每個資料點分配到最近的群中心，再反覆更新中心。它常見用途包括：分群探索、客戶分群、影像色彩壓縮、資料前處理。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

K-Means 是最常見的聚類方法之一，屬於非監督式學習。它沒有標準答案 y ，只有資料特徵 X 。模型的任務是把資料分成 K 群，使同一群內的資料盡量相似，不同群之間盡量分開。演算法一開始選擇 K 個群中心，然後反覆進行兩步：把每個資料點分配到最近的中心；再根據群內資料重新計算中心。這個過程持續到中心變化很小或達到迭代上限。

K-Means 很適合客戶分群。例如，企業可以用最近購買時間、購買頻率、平均消費金額、商品偏好等特徵，把客戶分成高價值忠誠客、偶爾購買客、沉睡客、價格敏感客等群組。模型本身只會給出群號，例如 0、1、2、3，分析者需要根據每群的平均特徵為群組命名。這點很重要：非監督式學習的結果需要人類解讀，群號本身沒有商業意義。

K 的選擇是 K-Means 的核心問題。 K 太小會把不同型態的資料混在一起； K 太大則會把自然群組切得太碎。常見方法包括 elbow method 與 silhouette score。Elbow method 觀察群內平方和隨 K 增加而下降的曲線，尋找下降幅度開始變小的轉折點；silhouette score 則衡量資料點與自己群內及其他群的距離關係。這些方法提供參考，但最終仍要回到業務可用性：分出的群是否能形成可行策略？

K-Means 對尺度與離群值敏感。若某個特徵數值範圍很大，例如年收入遠大於購買次數，它會主導距離計算，因此標準化通常是必要步驟。離群值也可能拉動群中心，使群組代表性變差。若資料群形狀不是圓形，而是長條形、月牙形或密度差異很大，K-Means 可能表現不佳，這時可考慮 DBSCAN、Gaussian Mixture 或階層式聚類。

學習 K-Means 的關鍵不是只會呼叫 `fit_predict`，而是能把分群結果轉成洞察。報告中應該呈現每群的資料筆數、平均特徵、代表性行為、可採取策略與風險。例如高價值客可以設計忠誠方案，沉睡客可以做喚回活動，新客可以做 onboarding。若只展示散佈圖與群號，沒有解讀與行動建議，分群就只是視覺效果，而不是資料分析成果。

範例情境

客戶分群：根據消費頻率、平均客單價、最近一次購買時間，把客戶分成不同營運群組。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 速度快，概念清楚，適合做分群入門與快速探索。
- 結果可用群中心解釋，方便觀察每一群的典型特徵。
- 可擴展到大量資料，常作為分群 baseline。

限制與注意事項

- 必須先決定 K，且初始中心會影響結果。
- 假設群形狀接近圓形且大小差異不大，對長條形或不規則群效果較差。
- 對尺度與離群值敏感，通常需要標準化與資料清理。

Python 入門範例

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)
model = KMeans(n_clusters=3, n_init='auto', random_state=42)
labels = model.fit_predict(X_scaled)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把 K-平均聚類寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用 K-平均聚類」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把 K-平均聚類與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，K-平均聚類不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

9. Principal Component Analysis, PCA（主成分分析）

章節定位：非監督式學習 / 降維。核心概念：把多個相關特徵轉換為少數主成分，用較少維度保留大部分資訊。
代表公式／規則：尋找能保留最大變異量的正交方向，常以 SVD 計算。

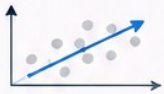
9 Principal Component Analysis (PCA) 主成分分析	非監督式學習	降維 / 簡化特徵 保留重要資訊	
---	--------	---------------------	---

圖 9-1 資訊圖表細部裁切：Principal Component Analysis, PCA（主成分分析）。

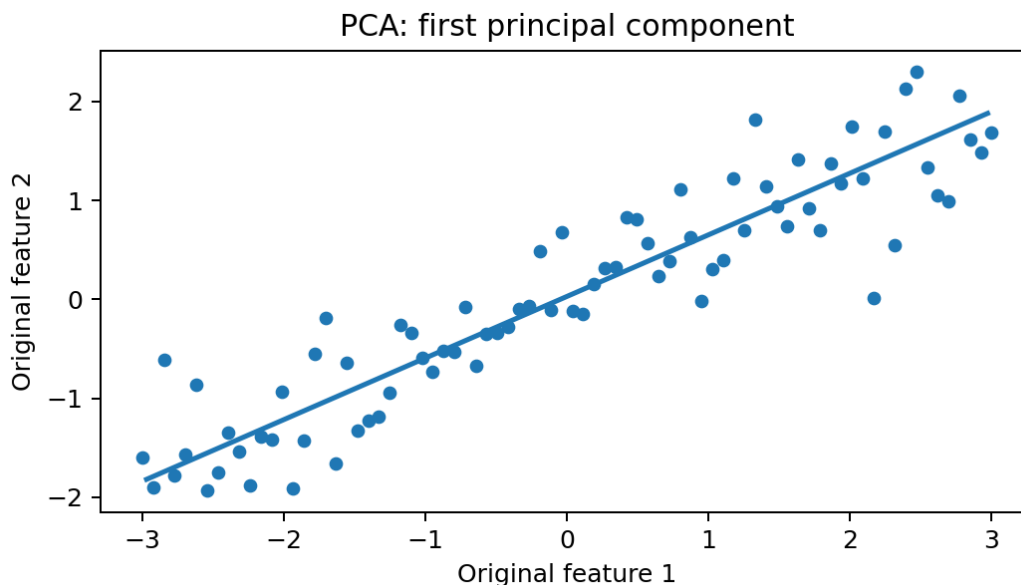


圖 9-2 主成分分析概念示意圖。

核心直覺與學習目標

主成分分析的任務可以簡化理解為：把多個相關特徵轉換為少數主成分，用較少維度保留大部分資訊。它常見用途包括：降維、視覺化、去除共線性、資料壓縮、模型前處理。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

PCA 主成分分析是常用的降維方法。當資料有很多特徵時，特徵之間可能高度相關，例如身高、體重、BMI，或不同消費欄位之間存在共同趨勢。PCA 會尋找新的座標軸，使第一個主成分保留最多變異，第二個主成分在與第一個正交的前提下保留次多變異，以此類推。最後，我們可以用較少主成分表示原資料，達到壓縮與視覺化效果。

PCA 的直覺可以想像成拍照角度。若一團資料在二維平面中沿著斜方向拉長，從原本的 x 軸或 y 軸看都不一定最有效；PCA 會找出資料拉伸最明顯的方向，將它視為第一主成分。若要把資料降到一維，就把資料投影到這條方向上。這樣雖然會失去部分資訊，但能保留最大變異。若降到二維，常可用於視覺化高維資料，觀察群組或離群點。

在機器學習流程中，PCA 常用於資料探索、降噪、降低共線性與加速模型。有時候原始特徵很多，模型訓練速度慢或容易過度擬合，PCA 可先把特徵壓縮到較少維度。也可以在 K-Means 前使用 PCA，把高維資料投影到主要方向，再做分群與視覺化。不過，PCA 不是萬能的特徵選擇。它保留的是變異最大方向，不一定是對目標 y 最有預測力的方向，因為 PCA 本身不知道標準答案。

PCA 對尺度非常敏感。如果某個特徵的數值範圍很大，它可能主導主成分方向。因此，使用 PCA 前通常要標準化。另一個限制是可解釋性：主成分是原始特徵的線性組合，不像原欄位那樣直覺。例如第一主成分可能同時包含收入、消費額、購買頻率等權重，解釋時需要查看 loading，才能說明它大致代表「整體消費強度」或其他潛在概念。

學習 PCA 時，建議觀察 `explained_variance_ratio_`，它告訴我們每個主成分保留多少變異。若前兩個主成分只保留 35% 變異，用二維圖解讀資料就要保守；若前兩個主成分保留 85%，二維圖就比較有代表性。報告中可以畫累積解釋變異曲線，說明選擇多少主成分。這比只說「我把資料降到 2 維」更有說服力。

範例情境

資料視覺化：把數十個客戶行為特徵降到 2 維，以便觀察是否存在自然群集。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 能減少特徵數、降低雜訊與共線性。
- 主成分可依解釋變異比例排序，方便評估保留多少資訊。
- 常用於高維資料探索與視覺化。

限制與注意事項

- 主成分是原特徵的線性組合，直覺解釋較困難。
- 只能捕捉線性結構；非線性流形需要其他方法。
- 對尺度敏感，通常要先標準化。

Python 入門範例

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把主成分分析寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用主成分分析」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把主成分分析與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時

間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，主成分分析不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

10. Neural Networks / Deep Learning (神經網路／深度學習)

章節定位：監督式與非監督式皆可。核心概念：利用多層節點與非線性轉換，從大量資料中學習複雜表徵。代表公式／規則：多層函數組合： $\hat{y} = f_L(\dots f_2(f_1(xW_1 + b_1))\dots)$

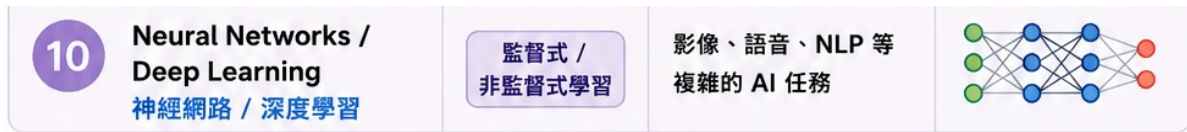
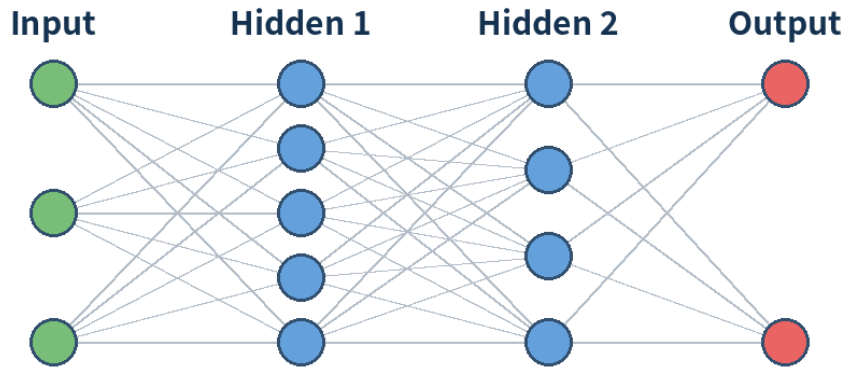


圖 10-1 資訊圖表細部裁切：Neural Networks / Deep Learning（神經網路／深度學習）。



Neural network: 多層節點透過權重與非線性函數學習複雜模式。

圖 10-2 神經網路／深度學習概念示意圖。

核心直覺與學習目標

神經網路／深度學習的任務可以簡化理解為：利用多層節點與非線性轉換，從大量資料中學習複雜表徵。它常見用途包括：影像、語音、自然語言、生成式 AI、複雜非結構化資料。本節會從資料、模型、預測、評估四個角度理解它，而不是只背公式。

神經網路是一類由多層節點組成的模型，每一層會對輸入做線性轉換，再經過非線性 activation function，最後形成輸出。若只有輸入層、少數隱藏層與輸出層，可以稱為多層感知器 MLP；若模型很深、資料量很大、用於影像或文字等複雜任務，通常稱為深度學習。神經網路的重要性在於它能學習複雜非線性函數，並能自動形成中間表徵。

在影像任務中，早期層可能學到邊緣與紋理，中間層學到局部形狀，高層學到物件部件或語意概念。在自然語言處理中，模型會把文字轉成向量表示，捕捉詞義、句法與上下文。Transformer 架構進一步透過 attention 機制處理長距離依賴，成為大型語言模型與生成式 AI 的核心之一。雖然本報告的重點是十大常見演算法，但理解神經網路能幫助你連接傳統機器學習與現代 AI。

神經網路的訓練通常使用反向傳播與梯度下降。模型先做 forward pass 產生預測，計算 loss，再把誤差往回傳，更新各層權重。學習率、batch size、層數、每層神經元數、activation、regularization、optimizer 都會影響結果。這也是神經網路比傳統模型難上手的原因之一：它不是只有一個模型名稱，而是一整套架構設計與訓練策略。

神經網路的優勢是表徵能力強，特別適合影像、語音、文字、時間序列、生成模型等複雜資料。但它也有明顯代價：需要更多資料與算力，訓練時間較長，模型可解釋性較弱，調參空間大。若資料只有幾百筆表格資料，直接使用神經網路未必比隨機森林好；若資料是大量圖片或長文字，傳統模型可能需要大量人工特徵工程，而神經網路能自動學習表徵。

對初學者而言，學習神經網路時應先建立三個觀念：第一，神經網路仍然是在做函數逼近，不是魔法；第二，資料品質比模型架構更重要，錯誤標籤與偏差資料會讓模型學到錯誤模式；第三，評估與監控不可省略，尤其生成式 AI 可能出現幻覺、偏見與不可預期輸出。在實務中，深度學習常與傳統模型搭配使用，例如用神經網路抽取文字或影像特徵，再用傳統分類器完成特定任務。

範例情境

影像辨識與 NLP：CNN 可用於圖片分類，Transformer 可用於文字生成、摘要、問答。

在實作時，請先定義輸入特徵、目標變數、訓練資料來源與評估方法。若是監督式學習，要確認標準答案是否可靠；若是非監督式學習，要思考結果如何被命名與驗證；若是深度學習，要確認資料量、算力與模型監控是否足夠。

優點

- 可學習高度非線性與複雜模式。
- 可處理影像、語音、文字等非結構化資料。
- 大型資料與運算資源充足時，表現上限很高。

限制與注意事項

- 需要較多資料、算力與調參經驗。
- 可解釋性較弱，除錯與信任建立較困難。
- 訓練流程受初始化、學習率、正則化、資料品質影響很大。

Python 入門範例

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

程式範例只呈現最基本呼叫方式。正式專題還需要加入資料切分、前處理、交叉驗證、評估指標與錯誤分析。若模型需要標準化，建議使用 Pipeline，避免訓練資料與測試資料處理不一致。

延伸實作建議與報告寫法

若要把神經網路／深度學習寫成作業或專題的一個章節，建議先用一小段話交代資料背景，再用一張圖展示資料分布或模型結果，接著說明模型假設、訓練設定與評估指標。不要只貼程式碼，因為程式碼只能證明你有執行模型，不能證明你理解模型。更好的寫法是說明「為什麼這個問題適合用神經網路／深度學習」、「資料前處理如何影響模型」、「模型結果如何轉成決策」。這樣的章節會比較像完整研讀報告，而不是 API 使用筆記。

在實驗設計上，可以把神經網路／深度學習與另一個 baseline 比較。例如監督式模型可以比較訓練集與測試集分數，觀察是否過度擬合；非監督式模型可以比較不同參數下的分群或降維結果；神經網路則可以比較不同層數、學習率或正則化設定。比較模型時要保持資料切分一致，否則分數差異可能來自資料，而不是模型能力。若有時間，建議加入錯誤案例分析，例如列出預測錯誤最大的樣本、最容易混淆的類別、或每一群中最不典型的資料點。

從學習角度看，神經網路／深度學習不應被孤立背誦，而要放進完整機器學習流程。資料取得後，要先做資料理解與清理；建模前，要思考特徵是否有意義；建模後，要用合適指標評估；報告時，要把結果用一般人能理解的語言說明。若你的目標是求職或課程專題，建議在 GitHub README 中放入問題定義、資料欄位、模型流程圖、主要圖表、結論與改進方向。這會比單純上傳 notebook 更能呈現你的資料分析能力。

附錄 A：如何選擇演算法

選擇演算法時，先不要從模型名稱開始，而是從問題開始。若答案是連續數值，例如價格、銷售量、耗電量，先考慮回歸模型；若答案是類別，例如是否流失、是否詐欺、文章類別，先考慮分類模型；若沒有答案，只想探索資料結構，考慮聚類或降維；若資料是影像、語音、文字等非結構化資料，則需要特徵工程或深度學習方法。

第二步是看資料量與特徵型態。資料量少而需要解釋時，可從線性迴歸、邏輯迴歸、決策樹開始；資料量中等且特徵互動複雜時，隨機森林常是強 baseline；高維文字特徵可嘗試 Naive Bayes、邏輯迴歸與線性 SVM；分群探索可嘗試 K-Means；高維資料視覺化可嘗試 PCA。不要一開始就追求最複雜模型，因為 baseline 的意義是讓你知道複雜模型是否真的改善。

第三步是看評估與應用成本。某些任務錯判正類與錯判負類成本不同，例如醫療篩檢、詐欺偵測、資安告警。這時不能只看 accuracy，要看 precision、recall、F1、ROC-AUC、PR-AUC，並調整決策門檻。回歸任務也要選擇合理指標，例如 MAE 容易解釋，RMSE 對大錯誤更敏感， R^2 可看整體解釋比例但不代表業務價值。

第四步是看可解釋性需求。若需要向主管、客戶、審查單位說明模型原因，線性模型、邏輯迴歸與淺層決策樹會比較容易。若追求準確率且可接受較複雜解釋，隨機森林、SVM 或神經網路可能更合適。實務中常用兩段式策略：先用可解釋模型建立理解，再用更強模型追求效能，最後用模型解釋工具輔助溝通。

演算法選擇速查表

問題狀況	建議先試	原因
預測連續數值	Linear Regression 、 Random Forest Regressor	先用線性模型建立解釋，再用森林捕捉非線性。
二元分類且需要機率	Logistic Regression	輸出機率，便於門檻調整與風險排序。
規則要好解釋	Decision Tree	樹狀規則容易視覺化與溝通。
表格資料想要強 baseline	Random Forest	穩定、能處理非線性與特徵交互。
高維文字分類	Naive Bayes 、 Linear SVM	對稀疏高維特徵常有良好效率與效果。
小資料、相似度明顯	KNN	依附近樣本做判斷，直覺簡單。
沒有標籤要分群	K-Means	快速探索資料群組。
特徵太多要視覺化	PCA	保留主要變異並降到 2D 或 3D。
影像、語音、長文字	Neural Networks / Deep Learning	表徵能力強，適合非結構化資料。

表 A-1 演算法選擇速查表。

附錄 B：評估指標與常見陷阱

回歸任務常用 MAE、MSE、RMSE、 R^2 。MAE 是平均絕對誤差，單位與目標相同，容易向非技術人員解釋；RMSE 會放大大錯誤，適合重視大偏差的情境； R^2 表示模型相對於只猜平均值改善多少，但 R^2 高不一定代表商業可用，仍要看誤差是否在可接受範圍內。

分類任務常用 accuracy、precision、recall、F1、ROC-AUC、PR-AUC。accuracy 在類別平衡時直覺，但在類別不平衡時容易誤導。Precision 表示模型預測為正的樣本中有多少是真的正；Recall 表示真正的正類中有多少被找出來。F1 是 precision 與 recall 的調和平均，常用於需要平衡兩者的情境。

資料洩漏是初學者常犯錯誤。舉例來說，若先對全部資料做標準化，再切分訓練集與測試集，測試資料的統計資訊就洩漏到訓練流程。正確方式是在訓練集 fit 標準化器，再用同一個轉換器 transform 測試集。使用 scikit-learn Pipeline 可以大幅降低這類錯誤。

過度擬合是另一個常見問題。模型在訓練資料上表現很好，但在測試資料上表現差，代表它可能記住資料細節而不是學到一般規律。控制方式包括簡化模型、正則化、剪枝、增加資料、交叉驗證、資料增強或早停。不同模型有不同控制參數，例如決策樹的 max_depth、邏輯迴歸的 C、神經網路的 dropout 與 learning rate。

最後，模型評估要連回業務問題。若模型要用於客戶流失預警，重點可能不是總 accuracy，而是能否提前找出高風險客戶並提升留存率。若模型要用於推薦系統，離線準確率只是第一步，還要觀察點擊率、轉換率、使用者體驗與公平性。機器學習報告最重要的不是模型分數本身，而是模型能否支援更好的決策。

參考資料

- scikit-learn User Guide：Supervised learning、Unsupervised learning、Model selection and evaluation。
- scikit-learn Linear Models：線性模型與邏輯迴歸相關說明。
- scikit-learn Support Vector Machines：SVM 分類、回歸、kernel 與實務提醒。
- scikit-learn Decision Trees 與 Ensembles：決策樹、隨機森林與集成方法。
- scikit-learn Nearest Neighbors：監督式與非監督式鄰近方法。
- scikit-learn Naive Bayes：貝葉斯定理與條件獨立假設。
- scikit-learn Clustering：K-Means 群中心、inertia 與迭代流程。
- scikit-learn PCA / Decomposition：PCA 以 SVD 投影到低維空間。
- scikit-learn Neural Network Models：MLP 監督式神經網路、優缺點與特徵尺度敏感性。

說明：本報告為中文學習整理，內容以教學與專題入門為目的；正式研究或專案請依最新官方文件、教材與實驗結果交叉驗證。

字數與圖表統計

本報告正文、表格與章節說明約含 20,304 個中文漢字，超過 20,000 字要求。文件包含 1 張完整資訊圖表、10 張資訊圖表細部裁切圖、10 張概念示意圖、2 張分類／總結圖或表格，以及多個程式範例。